Challenge Question

There is one compile-time error and two runtime inefficiencies in this code. How would you correct them?

```
#include <vector>
struct one {
    one( size_t len ) : _data(len) {
        for( size_t i=0; i<len; ++i ) {
            _data[i] = rand();
        }
    }
    int get( size_t index ) { return _data.at(index); }
    std::vector<int> _data;
};
double sum( const double array[], size_t len ) {
    double s=0;
    for( size_t i=0; i<len; ++i ) s += array.at(i);
    return s;
}
double sigma( const std::vector<double>& vec ) {
    double s=0;
    for( size_t i=0; i<vec.size(); ++i ) s += vec.at(i);
    return s;
}
// the dot product of (mathematical) vectors x and y is
// the sum of their componentwise products
double dot( double x[], size_t x_len, const std::vector<double>& y ) {
    double s=0;
    for( size_t i=0; i<x_len; ++i ) s += x[i]*y.at(i);
    return s;
}
```

(Solution on next page)

(Solution)

```
#include <vector>
struct one {
    one( size_t len ) : _data(len) {
        for( size_t i=0; i<len; ++i ) {
            _data[i] = rand();
        }
    }
    /***
     * this is a proper use of .at, as index is provided by the caller
     * and not validated as being < _data.size() before its use.
     */
    int get( size_t index ) { return _data.at(index); }
    std::vector<int> _data;
};
double sum( const double array[], size_t len ) {
    double s=0;
    /***
     * compile error, C arrays do not have member functions or variables
     * corrected below with []
     */
    for( size_t i=0; i<len; ++i ) s += array[i];
    return s;
}
double sigma( const std::vector<double>& vec ) {
    double s=0;
    /***
     * a runtime inefficiency, the for loop construction gaurantees
     *   0<=i<vec.size()
     * so there is no reason to if conditional logic (inside .at) executed
     * every iteration of the loop.
     * YES, this approach will make some of your 220 projects fail their timing tests.
     * instead just do   s += vec[i]
     */
    for( size_t i=0; i<vec.size(); ++i ) s += vec.at(i);
    return s;
}
// the dot product of (mathematical) vectors x and y is
// the sum of their componentwise products
double dot( double x[], size_t x_len, const std::vector<double>& y ) {
    double s=0;
    /***
     * similar to sigma(), we don't want conditional logic inside a for loop.
     * instead, verify that  x_len <= y.size() with one if statement _before_
     * the for loop, and use  s += x[i]*y[i]  in the for loop.
     */
    for( size_t i=0; i<x_len; ++i ) s += x[i]*y.at(i);
    return s;
}
```